

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Delimitación de parámetros posicionales estrictos mediante el símbolo / con la especificación del PEP 570.

Vol 6: Python Track

6x00

CULTURA HACKER Y WEB

Comunidad Python

Ratificación del PEP 13, reemplazando la monarquía benevolente de Guido por un Steering Council de 5 personas elegidas democráticamente.

Vol 6: Python Track

6x01

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Estandarización de genéricos de tipado en colecciones nativas (list[str], dict[str, int]) sin importar typing vía PEP 585.

Vol 6: Python Track

6x02

CULTURA HACKER Y WEB

Comunidad Python

Lanzamiento del primer taller de Django Girls en la conferencia EuroPython en Berlín por dos programadoras polacas.

Vol 6: Python Track

6x03

LENGUAJES Y RUNTIMES

Ecosistema Rust

Creación de Maturin, la herramienta de construcción que permite compilar y empaquetar módulos de Python implementados en Rust (PyO3).

Vol 6: Python Track

6x04

LENGUAJES Y RUNTIMES

CPython Runtime

Inclusión del compilador JIT experimental (Tier 2 Copy-and-Patch) en Python 3.13 para acelerar el bytecode en caliente.

Vol 6: Python Track

6x05

Brett Cannon, Guido van Rossum et al.

2018

Vol 6: Python Track

Fijó un mecanismo constitucional para evitar vacíos de poder, garantizando que Python evolucione sin personalismos.

6x01

Larry Hastings y Pablo Galindo

2019

Vol 6: Python Track

Permitió diseñar APIs seguras donde los nombres de argumentos internos pueden alterarse sin romper el código consumidor.

6x00

Ola Sitarska y Ola Sendecka

2014

Vol 6: Python Track

Con un tutorial claro y sin jerga, impulsó a más de 20,000 mujeres a desplegar su primera web en un fin de semana.

6x03

Łukasz Langa

2019

Vol 6: Python Track

Eliminó la molesta duplicación entre tipos nativos y alias en mayúsculas como `typing.List` o `typing.Dict`, simplificando las anotaciones.

6x02

Brandt Bucher y Faster CPython Team

2024

Vol 6: Python Track

Compila plantillas de código máquina nativo eliminando la sobrecarga del bucle central del evaluador de instrucciones.

6x05

Konstantin Sitnikov (PyO3 Team)

2020

Vol 6: Python Track

Facilitó que cualquier programador de Rust publicara paquetes nativos en PyPI con la misma simplicidad que un paquete de Python puro.

6x04

CULTURA HACKER Y WEB

Comunidad Python

Institucionalización de los Development Sprints presenciales de varios días tras finalizar las conferencias de PyCon.

6x06

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Implementación de ámbitos léxicos anidados (nested lexical scopes) resolviendo la regla clásica LEGB mediante el PEP 227.

6x07

LENGUAJES Y RUNTIMES

CPython Runtime

Aprobación histórica de la propuesta Free-threaded CPython para eliminar el GIL del núcleo del lenguaje mediante el PEP 703.

6x08

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Establecimiento de la jerarquía numérica abstracta (Number, Real, Integral) en el módulo numbers vía PEP 3141.

6x09

LENGUAJES Y RUNTIMES

CPython Runtime

Estandarización del protocolo de llamadas ultrarrápido Vectorcall en CPython a través del PEP 590.

6x0A

CULTURA HACKER Y WEB

Comunidad Python

Celebración del histórico primer taller de usuarios (First Python Workshop) en las instalaciones del instituto NIST en Maryland.

6x0B

Jeremy Hylton

2001

Vol 6: Python Track

*Requirió inicialmente from
__future__ import
nested_scopes, desatando
debates entre partidarios del
esquema plano anterior.*

6x07

PyCon Committee

2005

Vol 6: Python Track

*Sentar a novatos y veteranos
codo a codo en sprints impulsó
el cierre de bugs y forjó
futuros core developers.*

6x06

Jeffrey Yasskin

2007

Vol 6: Python Track

*Empleó clases base abstractas
(ABCs) para permitir que
librerías externas como NumPy
interactuaran con los tipos
nativos.*

6x09

Sam Gross (Meta)

2023

Vol 6: Python Track

*Tras décadas de resistencia,
el Steering Council aceptó
hacer del modo sin GIL el
futuro oficial de CPython.*

6x08

NIST (Guido van Rossum)

1994

Vol 6: Python Track

*Apenas asistieron 20 pioneros
para conocer a Guido; casi la
mitad trabajaba en agencias y
laboratorios de EE.UU.*

6x0B

Mark Shannon y Jeroen
Demeyer

2019

Vol 6: Python Track

*Evitó la costosa creación de
tuplas y diccionarios
intermedios temporales cada
vez que se invocaba una
función con argumentos
posicionales o de clave.*

6x0A

LENGUAJES Y RUNTIMES

CPython Runtime

Sustitución del histórico parser LL(1) por un nuevo motor PEG (Parsing Expression Grammar) en el PEP 617 para Python 3.9.

Vol 6: Python Track

6x0C

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Introducción de la función reversed() para iteración inversa sin mutar la secuencia original bajo el PEP 322.

Vol 6: Python Track

6x0D

LENGUAJES Y RUNTIMES

CPython Runtime

Aparición de Shed Skin, un compilador experimental que traducía programas en Python tipado estricto a código C++ nativo de alto rendimiento.

Vol 6: Python Track

6x0E

CULTURA HACKER Y WEB

Comunidad Python

Votación histórica de la PSF para prorrogar el fin de ciclo de vida (EOL) de Python 2.7 por cinco años adicionales.

Vol 6: Python Track

6x0F

LENGUAJES Y RUNTIMES

CPython Runtime

Anuncio oficial de Mojo, lenguaje que combina la sintaxis de Python con rendimiento directo en hardware y aceleradores de ML.

Vol 6: Python Track

6x10

LENGUAJES Y RUNTIMES

Ecosistema Rust

Creación de virtualenv, permitiendo aislar entornos de librerías independientes por proyecto sin contaminar el sistema operativo.

Vol 6: Python Track

6x11

Raymond Hettinger

2004

Vol 6: Python Track

Sustituyó el costoso truco de clonar e invertir listas en memoria mediante la notación de rebanado inverso `seq[::-1]`.

6x0D

Guido van Rossum, P. Galindo y Nikolaou

2020

Vol 6: Python Track

Permitió sintaxis complejas con múltiples miradas adelante (lookahead), base del `match/case` de Python 3.10.

6x0C

Python Software Foundation

2014

Vol 6: Python Track

La directiva reconoció que retirar Python 2 en 2015 fracturaría la industria dada la lenta migración a Python 3.

6x0F

Mark Dufour

2005

Vol 6: Python Track

Demostró aceleraciones de hasta 200x en matemáticas puras, anticipando el interés de la industria por dialectos compilados.

6x0E

Ian Bicking

2007

Vol 6: Python Track

Modificaba rutas copiando symlinks del binario de Python a carpetas locales, erradicando los temidos conflictos entre versiones de paquetes globales.

6x11

Modular (Chris Lattner)

2023

Vol 6: Python Track

Creado por el autor de LLVM y Swift, promete acelerar código Python hasta 35,000x combinando tipado y GPUs.

6x10

LENGUAJES Y RUNTIMES

Frameworks Web

Debut de Django REST Framework (DRF), brindando serializadores avanzados y vistas navegables para construir APIs RESTful profesionales.

6x12

Vol 6: Python Track

LENGUAJES Y RUNTIMES

CPython Runtime

Introducción del recolector de basura generacional para ciclos de referencias huérfanas en Python 2.0.

6x13

Vol 6: Python Track

LENGUAJES Y RUNTIMES

Ecosistema Rust

Lanzamiento de Ruff Format, ofreciendo un reemplazo directo y drop-in para Black programado íntegramente en Rust.

6x14

Vol 6: Python Track

LENGUAJES Y RUNTIMES

Frameworks Web

Nacimiento de Twisted, framework asíncrono por eventos antes de existir corrutinas nativas en Python.

6x15

Vol 6: Python Track

LENGUAJES Y RUNTIMES

CPython Runtime

Rediseño radical del paso de hilos en el núcleo con la llegada del New GIL en el intérprete de CPython.

6x16

Vol 6: Python Track

LENGUAJES Y RUNTIMES

Ecosistema Rust

Consagración de flake8, uniendo PyFlakes, el verificador de estilo pep8 y el medidor de complejidad ciclomática McCabe en una sola CLI.

6x17

Vol 6: Python Track

Neil Schemenauer

2000

Vol 6: Python Track

Añadió tres generaciones de recolección cíclica, evitando fugas de memoria sin degradar el conteo de referencias.

6x13

Tom Christie

2011

Vol 6: Python Track

Su interfaz web interactiva para explorar endpoints y enviar payloads al vuelo transformó el desarrollo de APIs backend.

6x12

Glyph Lefkowitz

2001

Vol 6: Python Track

Creado para un videojuego online, sus objetos Deferred inspiraron las promesas asíncronas modernas en JavaScript.

6x15

Astral

2023

Vol 6: Python Track

Replica el formato de Black con 99% de fidelidad, formateando repositorios enteros en microsegundos.

6x14

Tarek Ziadé

2010

Vol 6: Python Track

Fue el juez inflexible de estilo que cuidó millones de repositorios antes de los formateadores automáticos.

6x17

Antoine Pitrou

2010

Vol 6: Python Track

Reemplazó el conteo de 100 bytecodes por un temporizador (switch_interval), erradicando batallas destructivas entre hilos.

6x16

LENGUAJES Y RUNTIMES

Frameworks Web

Lanzamiento de uvloop, una reimplementación del bucle de eventos de asyncio programada en Cython sobre la biblioteca en C libuv.

6x18

LENGUAJES Y RUNTIMES

Ecosistema Rust

Lanzamiento del nuevo algoritmo de resolución de dependencias con retroceso (backtracking resolver) en pip 20.3.

6x19

LENGUAJES Y RUNTIMES

Ecosistema Rust

Lanzamiento de Pipenv, combinando Pipfile.lock y entornos virtuales automáticos bajo el lema "Dev Workflow for Humans".

6x1A

LENGUAJES Y RUNTIMES

CPython Runtime

Lanzamiento público de GraalPy, la implementación de Python para la máquina virtual políglota de alto rendimiento GraalVM.

6x1B

CULTURA HACKER Y WEB

Comunidad Python

Cancelación del formato presencial de PyCon US en Pittsburgh debido al estallido de la pandemia de COVID-19, pivotando a modalidad remota.

6x1C

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Creación del módulo orientado a objetos pathlib para gestión semántica de rutas de archivos en el PEP 428.

6x1D

Pradyun Gedam y Paul Moore

2020

Vol 6: Python Track

Eliminó el bug de pip que instalaba dependencias incompatibles si un paquete posterior pisaba al anterior.

6x19

**Yury Selivanov
(MagicStack)**

2016

Vol 6: Python Track

Demostró que un bucle en Python podía ser de 2 a 4 veces más rápido que Node.js, superando 100k requests/seg.

6x18

Oracle Labs

2021

Vol 6: Python Track

Logró interoperabilidad nativa sin costo (zero-overhead) entre Python, Java, JavaScript, Ruby y lenguajes compilados por LLVM.

6x1B

Kenneth Reitz

2017

Vol 6: Python Track

Respaldado por la PSF para emular la experiencia de npm, causó furor aunque luego sufrió lentitud en su resolver.

6x1A

Antoine Pitrou

2014

Vol 6: Python Track

Permitió manipular rutas mediante sobrecarga del operador de barra / (ruta / "archivo.txt"), jubilando las llamadas anidadas a os.path.

6x1D

**PSF Staff y Comité
Organizador**

2020

Vol 6: Python Track

Pese al impacto financiero de cancelar el evento físico, la versión online gratuita unió a la comunidad global.

6x1C

LENGUAJES Y RUNTIMES

Frameworks Web

Nacimiento de Celery, sistema distribuido de colas asíncronas para segundo plano con RabbitMQ y Redis.

6x1E

LENGUAJES Y RUNTIMES

Ecosistema Rust

Eliminación definitiva del módulo histórico distutils de la biblioteca estándar de CPython en Python 3.12.

6x1F

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Incorporación de las @dataclass para autogeneración de métodos especiales (__init__, __repr__) bajo el PEP 557.

6x20

CULTURA HACKER Y WEB

Comunidad Python

Lanzamiento del PSF Grants Program para financiar eventos, conferencias comunitarias y sprints locales en todo el mundo.

6x21

LENGUAJES Y RUNTIMES

CPython Runtime

Incorporación del intérprete adaptativo con especialización dinámica de instrucciones en el PEP 659 para Python 3.11.

6x22

CULTURA HACKER Y WEB

Comunidad Python

Fin de ciclo de soporte oficial y apagado (Sunset) de la rama Python 2.7 por el equipo de desarrollo de CPython.

6x23

**Victor Stinner y CPython
Core Team**

2023

Vol 6: Python Track

Tras una década de avisos con el PEP 632, forzó a la comunidad a migrar de setup.py hacia estándares en pyproject.toml.

6x1F

Ask Solem

2009

Vol 6: Python Track

Es el estándar para descargar tareas pesadas de emails, reportes y modelos fuera del ciclo de respuesta HTTP.

6x1E

Python Software Foundation

2007

Vol 6: Python Track

Ha repartido millones de dólares en becas para talleres, sprints de código y congresos en África, América Latina y Asia.

6x21

Eric V. Smith

2017

Vol 6: Python Track

Inspirado en la popular librería comunitaria attr, redujo toneladas de código repetitivo (boilerplate) en clases contenedoras de datos.

6x20

Python Software Foundation

2020

Vol 6: Python Track

Formalizado en el PEP 373; marcó la última versión 2.7.18 y culminó una transición hacia Python 3 que se extendió por más de una década.

6x23

Mark Shannon

2022

Vol 6: Python Track

Reemplaza bytecodes en caliente por variantes monomórficas según tipos observados, acelerando CPython hasta un 60%.

6x22

LENGUAJES Y RUNTIMES

CPython Runtime

Intento pionero de remover el GIL mediante el parche experimental Free Threading en Python 1.4.

Vol 6: Python Track

6x24

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Lanzamiento de Python 2.7, concebida como la versión de transición final y de soporte extendido de la rama 2.x.

Vol 6: Python Track

6x25

LENGUAJES Y RUNTIMES

Frameworks Web

Creación de Django Channels, extendiendo el corazón síncrono de Django para admitir WebSockets, chats y tareas en tiempo real.

Vol 6: Python Track

6x26

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Estandarización de cabeceras de codificación de archivos fuente con la directiva `# -- coding: utf-8 --` vía PEP 263.

Vol 6: Python Track

6x27

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Introducción de la sintaxis formal de anotaciones de tipos para variables de clase y módulo vía PEP 526.

Vol 6: Python Track

6x28

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Transformación de generadores en corrutinas completas mediante el método `.send()` y excepciones con el PEP 342.

Vol 6: Python Track

6x29

Benjamin Peterson

2010

Vol 6: Python Track

Su fin de vida previsto para 2015 tuvo que postergarse al 1 de enero de 2020 por el inmenso código legacy sin migrar.

6x25

Greg Stein

1999

Vol 6: Python Track

Logró hilos paralelos, pero bloquear cada operación duplicó el tiempo de ejecución monohilo, forzando su descarte.

6x24

**Marc-André Lemburg y
Martin von Löwis**

2003

Vol 6: Python Track

Evitó que el intérprete fallara con excepciones de sintaxis al encontrar acentos, tildes o caracteres asiáticos en comentarios y strings.

6x27

Andrew Godwin

2016

Vol 6: Python Track

Introdujo el concepto de capas de canales (channel layers) sobre Redis, permitiendo a Django cruzar la frontera de las arquitecturas HTTP clásicas.

6x26

Guido van Rossum y Phillip Eby

2005

Vol 6: Python Track

Permitió inyectar valores directamente en generadores en ejecución, sentando los cimientos teóricos para los futuros frameworks asíncronos.

6x29

Ryan Gonzalez y Guido van Rossum

2016

Vol 6: Python Track

Habilitó expresiones directas como primer_nombre: str = "Ada", allanando el camino para modelos declarativos como Pydantic.

6x28

LENGUAJES Y RUNTIMES

Ecosistema Rust

Expansión de uv como gestor integral de proyectos, scripts monosemilla y versiones de Python.

6x2A

LENGUAJES Y RUNTIMES

CPython Runtime

Creación del objeto inmortal (PEP 683), permitiendo compartir tipos entre núcleos sin mutar conteos de referencias.

6x2B

LENGUAJES Y RUNTIMES

Frameworks Web

Creación de aiohttp, proveyendo cliente y servidor HTTP asíncrono construido sobre el nuevo bucle de eventos asyncio de Python 3.4.

6x2C

LENGUAJES Y RUNTIMES

Frameworks Web

Lanzamiento del toolkit y microframework Starlette, concebido como el pilar fundamental para la nueva era de aplicaciones ASGI.

6x2D

LENGUAJES Y RUNTIMES

Ecosistema Rust

Nacimiento de Poetry, gestionando dependencias deterministas, resolución de grafos y empaquetado en pyproject.toml.

6x2E

LENGUAJES Y RUNTIMES

CPython Runtime

Debut de Pyrex, compilador precursor de extensiones en C mezclando sintaxis de Python con tipos estáticos.

6x2F

**Eric Snow y Eddie Elizondo
(Meta)**

2023

Vol 6: Python Track

*Permitió a Instagram/Meta
compartir gigabytes de memoria
entre procesos sin activar
Copy-on-Write accidentalmente.*

6x2B

Astral

2024

Vol 6: Python Track

*Permitió ejecutar scripts con
dependencias embebidas
directamente mediante uv run y
descargar versiones de CPython
a la carta en milisegundos.*

6x2A

Tom Christie

2018

Vol 6: Python Track

*Aportó manejo robusto de
WebSockets, middleware
asíncrono y streaming de datos
con cero dependencias externas
pesadas.*

6x2D

**Nikolay Kim y Andrew
Svetlov**

2013

Vol 6: Python Track

*Fue la primera gran librería
en demostrar el potencial de
las corrutinas para consumir y
exponer APIs asíncronas con
miles de conexiones abiertas.*

6x2C

Greg Ewing

2002

Vol 6: Python Track

*Permitió escribir extensiones
rápidas sin manipular
manualmente las complejas
macros de la Python C API.*

6x2F

Sébastien Eustace

2018

Vol 6: Python Track

*Su estricto resolver y su
elegante CLI en Python lo
convirtieron en el estándar
predilecto de la comunidad.*

6x2E

LENGUAJES Y RUNTIMES

Ecosistema Rust

Especificación de la interfaz estándar entre herramientas de empaquetado e instaladores mediante el PEP 517.

Vol 6: Python Track

6x30

LENGUAJES Y RUNTIMES

CPython Runtime

Creación de Stackless Python, desacoplando la pila de C para habilitar micro-hilos livianos (tasklets).

Vol 6: Python Track

6x31

LENGUAJES Y RUNTIMES

Frameworks Web

Liberación de Zope 2, servidor de aplicaciones pionero que introdujo la base de datos orientada a objetos ZODB.

Vol 6: Python Track

6x32

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Creación del tipo mutable de secuencias de bytes bytearray con la especificación del PEP 3137.

Vol 6: Python Track

6x33

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Introducción del método avanzado de formateo de cadenas str.format() a través de la especificación del PEP 3101.

Vol 6: Python Track

6x34

LENGUAJES Y RUNTIMES

CPython Runtime

Presentación pública de IronPython, llevando la sintaxis de Python al entorno de ejecución de Microsoft .NET y Mono.

Vol 6: Python Track

6x35

Christian Tismer

1998

Vol 6: Python Track

Fue el motor de EVE Online, permitiendo a sus servidores coordinar miles de naves espaciales en un único proceso.

6x31

Thomas Kluyver, Brett Cannon et al.

2019

Vol 6: Python Track

Desacopló definitivamente a instaladores como pip de backends de compilación como flit, hatch o poetry-core.

6x30

Guido van Rossum

2008

Vol 6: Python Track

Brindó manipulación de buffers binarios en memoria sin incurrir en las continuas copias e inmutabilidad del tipo bytes.

6x33

Jim Fulton (Digital Creations)

2000

Vol 6: Python Track

Guardaba objetos Python en disco sin usar SQL, impulsando las primeras aplicaciones web corporativas del lenguaje.

6x32

Jim Hugunin (Microsoft)

2004

Vol 6: Python Track

La demo en el CLR impresionó tanto a Microsoft que contrataron de inmediato a Hugunin para liderar el proyecto.

6x35

Talin

2006

Vol 6: Python Track

Diseñado para modernizar y eventualmente jubilar al venerable operador de interpolación heredado de C % (printf-style).

6x34

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Simplificación de la unión de tipos estáticos utilizando el operador de tubería `int | str` mediante el PEP 604.

6x36

LENGUAJES Y RUNTIMES

Sintaxis y PEPs

Adición de guiones bajos como separadores visuales de dígitos en literales numéricos (`1_000_000`) en el PEP 515.

6x37

LENGUAJES Y RUNTIMES

Frameworks Web

Lanzamiento del motor de plantillas Jinja2, optimizado para compilar código HTML directamente a código Python en memoria.

6x38

LENGUAJES Y RUNTIMES

Frameworks Web

Lanzamiento de Requests, la emblemática biblioteca cliente HTTP concebida bajo el lema HTTP for Humans.

6x39

LENGUAJES Y RUNTIMES

CPython Runtime

Reescritura del compilador de CPython introduciendo la estructura de Árbol de Sintaxis Abstracta (AST) formal mediante el PEP 339.

6x3A

LENGUAJES Y RUNTIMES

Ecosistema Rust

Nacimiento de Conda, el gestor de paquetes y entornos binarios multiplataforma creado para el ecosistema analítico y científico.

6x3B

Georg Brandl

2016

Vol 6: Python Track

Mejoró enormemente la legibilidad de máscaras binarias, direcciones hexadecimales y grandes números enteros sin alterar su valor en runtime.

6x37

Philippe Prados y Maggie Moss

2020

Vol 6: Python Track

Reemplazó la pesada y verbose sintaxis `Union[int, str]` de typing por una notación mucho más limpia y cercana a otros lenguajes modernos.

6x36

Kenneth Reitz

2011

Vol 6: Python Track

Reemplazó la farragosa y arcaica API de `urllib2`, convirtiéndose en el paquete de Python más descargado de la historia.

6x39

Armin Ronacher

2008

Vol 6: Python Track

Tomó la sintaxis `{{ variable }}` de Django, perfeccionando su velocidad y seguridad con auto-escape automático nativo.

6x38

Travis Oliphant y Peter Wang

2012

Vol 6: Python Track

Resolvió instalar librerías científicas con dependencias en Fortran y C sin compilar en Windows, macOS o Linux.

6x3B

Brett Cannon y Neil Schemenauer

2005

Vol 6: Python Track

Reemplazó el arcaico compilador de un solo paso por un diseño modular de dos fases que facilitó el análisis estático y la optimización de `bytecode`.

6x3A

Vol 6: Python Track

LENGUAJES Y RUNTIMES

CPython Runtime

Implementación original del cerrojo global del intérprete (Global Interpreter Lock o GIL) en el bucle central de CPython.

6x3C

Vol 6: Python Track

CULTURA HACKER Y WEB

Comunidad Python

Redacción del PEP 1, institucionalizando las propuestas de mejora como pilar de la evolución de Python.

6x3D

Vol 6: Python Track

LENGUAJES Y RUNTIMES

Ecosistema Rust

Creación de tox, la herramienta de orquestación de pruebas automatizadas sobre múltiples versiones de Python en entornos aislados.

6x3E

Vol 6: Python Track

CULTURA HACKER Y WEB

Comunidad Python

Formalización de The Zen of Python en el PEP 20, recopilando los 19 principios rectores de la filosofía del lenguaje.

6x3F

**Barry Warsaw y Jeremy
Hylton**

2000

Vol 6: Python Track

Inspirado en los RFCs de Internet, fijó que ningún cambio mayor ocurriría sin una propuesta pública documentada.

6x3D

Guido van Rossum

1992

Vol 6: Python Track

Evitaba carreras de memoria en C, sin prever el dolor de cabeza que causaría para el cómputo multinúcleo décadas después.

6x3C

Tim Peters

2004

Vol 6: Python Track

Incluido secretamente como huevo de pascua en el intérprete al ejecutar `import this`, compuesto como un poema de aforismos.

6x3F

Holger Krekel

2010

Vol 6: Python Track

Se convirtió en el guardián indiscutible de la integración continua (CI) para asegurar que una librería funcionara idénticamente en Python 2 y 3.

6x3E